Senior DevOps / SRE Interview Questions & Answers — Part 4

Advanced production-focused DevOps/SRE interview preparation for Lead/Staff-level roles.

---

101. How do you troubleshoot high disk IO in Kubernetes nodes?

Answer

I first determine whether the issue is:

- application-driven
- logging-related
- storage backend-related
- noisy neighbor problem

Commands:

```
iostat -xz 1
iotop
kubectl top node
```

Then I correlate:

- pod placement
- log volume
- database writes
- image pulls
- backup jobs

Mitigation:

- isolate workloads
- optimize storage class
- reduce sync writes
- improve caching
- move heavy workloads to dedicated nodes

---

102. Explain the difference between Deployment, StatefulSet, and DaemonSet.

Answer

Deployment

Used for stateless workloads.

Features:

- rolling updates
- replica management
- interchangeable pods

StatefulSet

Used for stateful applications.

Features:

- stable identity
- ordered startup
- persistent volumes

Examples:

- Kafka
- Cassandra
- MongoDB

DaemonSet

Runs one pod per node.

Used for:

- monitoring agents
- log collectors
- security agents

Choosing the wrong controller often creates operational problems.

---

103. How do you troubleshoot Kubernetes node pressure?

Answer

I identify the pressure type:

- memory pressure
- disk pressure
- PID pressure

Commands:

```
kubectl describe node
```

Then investigate:

- runaway pods
- excessive logs
- orphaned containers
- image buildup

Mitigation:

- cleanup
- scale nodes
- tune eviction thresholds
- rebalance workloads

Node pressure can cause cascading pod evictions.

---

104. Explain Kubernetes eviction policies.

Answer

Kubernetes evicts pods when node resources become constrained.

Common triggers:

- memory pressure
- disk pressure
- PID exhaustion

Eviction priority depends on:

- QoS class
- resource requests
- priority class

BestEffort pods are evicted first.

Critical workloads should have:

- Guaranteed QoS
- proper requests/limits

---

105. What is QoS class in Kubernetes?

Answer

QoS classes define pod scheduling and eviction priority.

Guaranteed

Requests = Limits

Highest stability.

Burstable

Requests defined but limits higher.

BestEffort

No requests/limits.

Lowest priority during resource pressure.

For production-critical workloads, I prefer Guaranteed or carefully tuned Burstable classes.

---

106. Explain Kubernetes operator pattern.

Answer

Operators extend Kubernetes using custom controllers.

They automate:

- deployment
- scaling
- backup
- failover
- lifecycle management

Examples:

- Prometheus Operator
- Elastic Operator
- ArgoCD

Operators are useful for complex stateful applications.

But poorly designed operators can overload the API server.

---

107. How do you debug failed init containers?

Answer

Init containers must complete successfully before app containers start.

Checks:

```
kubectl describe pod
kubectl logs <pod> -c <init-container>
```

Common failures:

- migration failures
- secret access
- network issues
- permission problems

I isolate whether the issue is startup dependency or application logic.

---

108. Explain Kubernetes affinity and anti-affinity.

Answer

Affinity controls pod placement preferences.

Node Affinity

Schedule on specific node groups.

Pod Affinity

Schedule near related pods.

Pod Anti-Affinity

Spread pods across nodes/zones.

I use anti-affinity heavily for HA workloads.

Without it, multiple replicas may land on the same node.

---

109. What happens when etcd becomes unavailable?

Answer

etcd stores Kubernetes cluster state.

If etcd fails:

- API server becomes unstable
- scheduling stops
- cluster state updates fail

Existing workloads may continue temporarily.

Recovery:

- restore etcd quorum
- recover snapshots
- validate consistency

Regular etcd backups are mandatory.

---

110. How do you back up and restore etcd?

Answer

Backup:

```
etcdctl snapshot save
```

Restore:

```
etcdctl snapshot restore
```

Best practices:

- encrypted backups
- scheduled snapshots
- restore testing
- offsite storage

A backup is useful only if restore works reliably.

---

111. Explain Kubernetes service types.

Answer

ClusterIP

Internal-only access.

NodePort

Exposes service on node ports.

LoadBalancer

Cloud provider-managed LB.

ExternalName

Maps to external DNS.

I prefer ingress + ClusterIP for most production architectures.

---

112. How do you debug pending PVCs?

Answer

Checks:

```
kubectl get pvc
kubectl describe pvc
```

Common causes:

- storage class missing
- insufficient storage
- CSI issues
- access mode mismatch

Storage failures often block pod scheduling entirely.

---

113. Explain CSI drivers.

Answer

CSI provides standardized storage integration.

Responsibilities:

- provisioning
- attachment
- snapshots
- expansion

Examples:

- EBS CSI
- Ceph CSI

- NFS CSI

CSI simplifies storage portability across environments.

---

114. How do you debug container networking across nodes?

Answer

I validate:

- pod IP reachability
- CNI routes
- overlay networking
- firewall rules
- MTU mismatch

Commands:

```
ip route
traceroute
```

Cross-node networking issues often involve:

- VXLAN
- security groups
- CNI misconfiguration

---

115. Explain container runtime differences.

Answer

Common runtimes:

- containerd
- CRI-O
- Docker (legacy in K8s)

Modern Kubernetes primarily uses:

- containerd

- CRI-O

I prefer lightweight runtimes with strong CRI integration.

---

116. How do you secure the Kubernetes API server?

Answer

Security controls:

- private endpoints
- RBAC
- audit logging
- network restrictions
- OIDC authentication
- MFA
- admission controllers

API server compromise can expose the entire cluster.

---

117. Explain admission controllers.

Answer

Admission controllers validate or mutate requests before persistence.

Examples:

- PodSecurity
- ResourceQuota
- LimitRanger
- OPA Gatekeeper

Use cases:

- enforce policies
- block insecure configs
- inject sidecars

Admission controllers are critical for governance.

---

118. What causes CrashLoopBackOff even when logs look normal?

Answer

Possible hidden causes:

- liveness probe failures
- dependency timeouts
- kernel signals
- OOMKilled
- startup race conditions

I always correlate:

- events
- kubelet logs
- metrics
- restart timestamps

Logs alone are not enough.

---

119. Explain ephemeral storage issues in Kubernetes.

Answer

Containers use ephemeral storage for:

- logs
- temp files
- writable layers

Problems occur when:

- logs grow uncontrollably
- applications generate temp data

This can trigger:

- disk pressure
- pod eviction

I monitor ephemeral storage usage carefully in production.

---

120. How do you troubleshoot slow API responses in microservices?

Answer

I trace the full request path.

Checks:

- tracing latency
- DB queries
- retries
- downstream dependencies
- thread pools
- connection pools

Common hidden causes:

- retry storms
- cache misses
- DNS latency

Distributed tracing is critical here.

---

121. Explain canary deployment rollback logic.

Answer

Rollback should be automated based on SLO degradation.

Metrics monitored:

- latency
- error rate
- saturation
- business SLIs

If thresholds exceed:

- traffic shifts back automatically

Automation reduces human reaction delay.

---

122. How do you debug memory fragmentation issues?

Answer

Memory fragmentation differs from memory leaks.

Symptoms:

- high RSS
- low usable memory
- allocation failures

I investigate:

- allocator behavior
- heap patterns
- runtime metrics

Tools:

- pprof
- jemalloc stats
- heap analyzers

---

123. Explain rate limiting strategies.

Answer

Rate limiting protects systems from overload.

Common approaches:

- token bucket
- leaky bucket
- fixed window
- sliding window

Used for:

- API protection
- abuse prevention
- fair usage

Good rate limiting prevents cascading failures.

---

124. How do you handle noisy alerts during outages?

Answer

I reduce noise aggressively.

Strategies:

- alert grouping
- suppression
- dependency-aware alerting
- inhibition rules

The goal is preserving engineer focus during incidents.

---

125. Explain horizontal pod autoscaler internals.

Answer

HPA periodically checks metrics and adjusts replicas.

Flow:

1. metrics collected
2. utilization calculated
3. desired replicas computed
4. scale action triggered

Dependencies:

- metrics server
- resource requests
- API server health

Scaling delays can happen due to stabilization windows.

---

126. How do you troubleshoot cluster autoscaler failures?

Answer

Checks:

- cloud permissions
- node group limits
- pending pods
- taints
- quotas

Common causes:

- max node count reached
- insufficient instance capacity
- wrong node selectors

Autoscaler debugging requires both Kubernetes and cloud understanding.

---

127. Explain platform engineering.

Answer

Platform engineering builds internal developer platforms.

Goals:

- self-service
- standardization
- reliability
- governance
- developer productivity

Platform teams reduce cognitive load for application teams.

---

128. How do you reduce deployment risk?

Answer

Strategies:

- canary releases
- feature flags
- automated rollback
- progressive delivery
- strong observability
- pre-production testing

Deployment safety is more about process than tooling.

---

129. Explain chaos engineering.

Answer

Chaos engineering intentionally introduces failures.

Purpose:

- validate resilience
- expose weak assumptions
- improve recovery readiness

Examples:

- node failures
- network latency
- DNS failures
- packet loss

Resilience cannot be assumed — it must be tested.

---

130. What is the biggest mindset shift from Senior to Staff Engineer?

Answer

Senior engineers optimize systems.

Staff engineers optimize organizations and engineering ecosystems.

Staff-level thinking includes:

- long-term reliability
- platform strategy
- organizational scalability
- standards and governance
- reducing operational toil
- enabling other engineers

The focus shifts from solving individual incidents to preventing entire categories of failures.